

Classes

CS 101 Intermediate Programming Spring 2026

Objectives

Covered:

1. [X] Procedural vs. Object-Oriented Programming
2. [X] Defining classes and objects
3. [X] Private and public access control
4. [X] Member functions
5. [X] Constructors and destructors
6. [X] Constructor and operator overloading
7. [X] Separation of interface and implementation

Other topics:

1. [] Arrays of Objects
2. [] Simulating dice (case study)
3. [] Unified Modeling Language (UML)
4. [] Instance and static members
5. [] Friends of Classes
6. [] Protected data members
7. [] Memberwise Assignment
8. [] Object Conversion
9. [] Aggregation
10. [] Class collaborations (case study)
11. [] Game simulation (case study)
12. [] Rvalue References and Move Semantics
13. [] Inheritance (derivative classes)
14. [] Polymorphism (share user interface at runtime)
15. [] Virtual Functions (overriding member functions at runtime)

Procedural vs. object-oriented programming (OOP) paradigms

- Most disciplines that grow over time are not just cobbled together: People identify patterns that are successful, explore and pursue them, and finally turn them into standards (until they fail).
- These patterns are also called "paradigms" (παράδειγματα). Two important programming paradigms are procedural and object-oriented programming.
- What does **procedural** mean? What does it achieve?

Centered creating procedures or functions. The main purpose is to create modules and reuse them over and over again.

- What is **object orientation**? What does it achieve?

OO means that everything is an object. And every object has its own state and behavior. Computation now happens by message (data) exchange between objects - like a conversation between agents.

Encapsulation of data (withhold some data), inheritance (of objects), polymorphism (operator overloading at compile-time).

Example: A light switch as message exchange between objects

- Example 1: A light switch.

```
#include <iostream>
using namespace std;

// Let there be light!
class Light {
public:
    void turnOn() { cout << "Light is on.\n"; }
    void turnOff() { cout << "Light is off.\n"; }
};

// Provide a switch!
class Switch {
    Light *light; // knows where to send the message
public:
    Switch(Light *l) : light(l) {}; // flips the switch
    void flip(bool on) {
        if (on)
            light->turnOn(); // sends 'turnOn' message to the Light
        else
            light->turnOff(); // sends 'turnOff' message to the Light
    }
};

// Test it by switching light on and off!
int main()
{
    Light lamp; // create a Light object called 'lamp'
    Switch s(&lamp); // create a Switch for the lamp
    s.flip(true); // throw the switch and turn lamp on
    s.flip(false); // throw the switch and turn lamp off
    return 0;
}
```

- Explanation:

The Switch does not directly manipulate the Light - it sends messages turnOn and turnOff. Each object controls its own behavior and data - this is a huge advantage when you want to solve more and more difficult problems.

- **Challenge (bonus assignment):**

What would the procedural version of this program look like?

Example: Procedural vs. object-oriented adding of objects

- What is **object-oriented** programming?

OOP is centered on creating and manipulating objects in a safe way:

- Example using an add function in procedural style:
 1. You may want to add integers.
 2. You may want to add floating-point numbers.

3. You may want to "add" (concatenate) strings.

- These are three different procedures. In C/C++, they can be **overloaded** so that each function is called add but which one is used depends on its arguments.
- Implementation:

```
#include <iostream>
using namespace std;

// Integer addition
int add(int a, int b)
{
    return (a+b);
}

// Floating-point addition
double add(double a, double b)
{
    return (a+b);
}

// String concatenation
string add(std::string a, std::string b)
{
    return (a+b);
}

// The main program calls each function
int main()
{
    cout << add(5,7) << endl; // addition of two integers
    cout << add(5.5,6.5) << endl; // addition of two floats
    cout << add("five","seven") << endl; // concatenation of two strings
    return 0;
}
```

- In the OOP style, a class named Adder has member functions that take care of both argument types:

```
#include <iostream>
using namespace std;

// The Adder class has 'public' methods (no data)
class Adder {
public:
    int add(int a, int b) // integer addition
    {
        return (a+b);
    }
    double add(double a, double b) // double addition
    {
        return (a+b);
    }
    string add(std::string a, std::string b) // string 'addition'
    {
        return (a+b);
    }
};

// The functions are called on the Adder object A
int main()
{
    Adder A;
    cout << A.add(5,7) << endl;
}
```

```
cout << A.add(5.5,6.5) << endl;
cout << A.add("five","seven") << endl;
return 0;
}
```

```
12
12
fiveseven
```

- Problems with Procedural Style
 - All add functions live in global scope.
 - Namespace pollution: more name conflicts.
 - Maintenance gets harder.
 - There are no objects and no states.
- Advantage of OOP
 - All add methods are now grouped in one class Adder.
 - Code is encapsulated (easier to read, find, reason about).
 - Behavior can be extended - reset, showState
 - You can associate behavior with state - count, lastResult.
- Criticisms of OOP
 - Significant overhead if you have class hierarchies.
 - The code can look more tangled
 - When the logic is simple then OOP adds needless structure.
 - Encapsulation can be illusion.
 - Tight coupling - classes are very interdependent
 - Testing can be difficult
- Why do "objects" and their "state" matter?
 - Object mimic real identities: BankAccount has data like balance and methods like deposit, withdraw, checkBalance.
 - Reuse objects and evolve them by extending your classes.
 - In UI (Amazon checkout) and in games, OOP rules.

An everyday "data encapsulation" example: The car

- What's part of the car's "user interface"? (What can you see/use?)

To use a car (as driver = user):

 - turn the wheel
 - open/close doors
 - use the gas pedal
 - brake
 - gear shift
 - display data on a dashboard
- What's part of the car's "internal implementation"? (What's hidden?)

Mechanisms hidden from the user (to be able to use the car):

- Steering column
- Fuel injection
- Brake system
- Most things outside of the driver's field of vision

Classes and objects

- What is a class? A class is three things: **keyword**, **code**, and **concept**.
- As a **keyword**:

It's a type that you can use to cast data - you can make your own categories. Light class allowed us to define a lamp.

- As **code**:

A class encapsulates the attributes (properties, member data), and behavior (member functions, methods) that any object of that class may have. When we define a lamp as a Light we can now turn it on/off.

```
// The Rectangle class objects have width and length
// You can set and get these properties for any Rectangle
class Rectangle {
private: // member data = properties
    double width;
    double length;
public: // member functions = behavior
    int test = 100;
    // setter / mutators
    void setWidth(double);
    void setLength(double);
    // getter / accessors
    double getWidth();
    double getLength();
    double getArea();
};
```

- As a **concept**:

A class describes an object like a blueprint

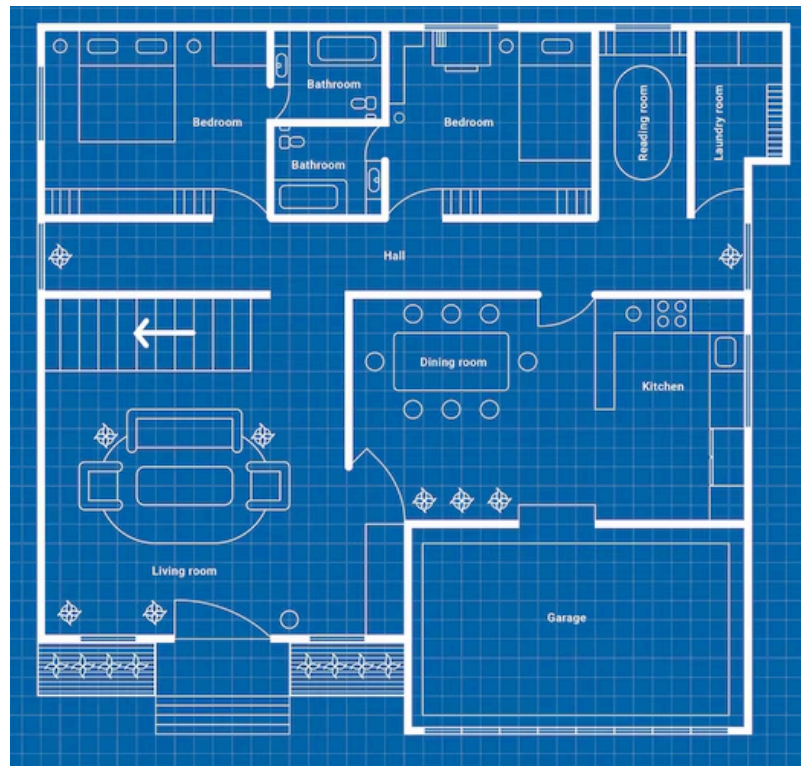


Figure 1: Blueprint of a house.

- The blueprint can be used to build any number of objects, or **instances**.



Figure 2: House objects build with a house blueprint.

- Example: What could you say about the Insect class and the two objects shown in the figure?

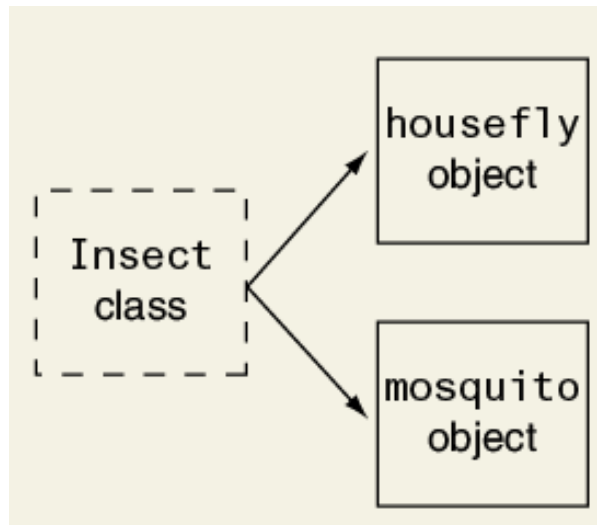


Figure 3: An Insect class and two of its instances.

- Answer:
 - Insect can describe attributes and behaviors of MANY insects.
 - housefly and mosquito and instantiated from Insect
 - mosquito members have a stinger and they sting
 - housefly members have a sucker and they suck

Using the string class

- You've already used C++ classes like vector and string.
- To use the string class you need to `#include <string>`. Now you can define string objects, and use string member functions.
- Example: Setting a city name, a State, and getting its length.

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string cityName;    // create a string class object
    cityName = "Batesville"; // assign string to it
    cout << cityName << endl;
    cout << cityName.length() << endl; // void string::length() {...}
    cout << cityName.append(", AR"); // string string::append(string) {...}
    return 0;
}
```

```
Batesville
10
Batesville, AR
```

- What about the time complexity of the `length` and `append` functions?

Operation	Time	Notes
<code>std::string::length</code>	$O(1)$	Length is stored internally at compile time
<code>std::string::append</code>	$O(1)$	Places new characters in allocated space

Introduction to classes (BankAccount)

Let's use a **bank account** as an example (since everyone has got one).

- What are the computing **procedures** to run a **bank account**?
 - get the account balance (getter)
 - deposit an amount (setter)
 - withdraw an amount (setter)

All of these are public: this means that they are available to any bank account object = It is what a client can do with a bank account.

- What are the **data** of a bank account?
 - current balance (private feature/attribute)
 - amount to be deposited (function argument)
 - amount to be withdrawn (function argument)

balance is private because we want to ensure that only our functions (validated transactions) are allowed to see it.

- What does designing a BankAccount class mean?
 - Write BankAccount procedures that manage the data
 - Create objects in main ("instantiating"=creating an instance)
 - Use the BankAccount objects in main

- What's an **object**?

To the class, an object is an instance of what the class represents - a lamp is a Light object, a savingsAccount is a BankAccount object etc.

- What's a **constructor**? Example?

A constructor creates objects of a chosen class. In the case of BankAccount, we are going to have a constructor that creates an account with a starting balance.

OOP workflow and class template

- How to do it in practice:

1. write a class = data + methods = description/blueprint
2. create instances of the class (main) = objects
3. use class methods on the class data = OOP

- General template:

```
class ClassName {  
  
    private:  
    // data members (attributes/fields/features)  
  
    public:  
    // member functions (methods) - Constructors, getter, setter, Destructor  
  
};
```

- Example: Code along using [OneCompiler.Com](https://onecompiler.com). Insert this code between the namespace statement and the main program, like a function. Note: All of the member functions are defined **inline**.

```
#include <iostream>  
using namespace std;  
  
class BankAccount  
{  
private:  
    double balance;  
public:  
    BankAccount(double startingBal) // constructor  
    { balance = startingBal; }  
  
    double getBalance() const // getter method/member function  
    { return balance; }  
  
    void deposit(double amount)  
    { balance += amount; }  
  
    void withdraw(double amount)  
    { balance -= amount; }  
};  
  
int main() {  
  
    // Create BankAccount  
    BankAccount account(200.);  
  
    // Get the balance  
    cout << "Starting balance: $" << account.getBalance();  
  
    // Deposit amount  
    account.deposit(300.);  
    cout << "\nDeposited $300. New balance: $" << account.getBalance();  
  
    // Withdraw amount  
    account.withdraw(100.);  
    cout << "\nWithdrew $100. New balance: $" << account.getBalance();  
  
    return 0;  
}
```

Member functions and member data encapsulation

- There are some similarities between defining/using functions (procedures) and classes.
- What if we do not define a member function inside the class definition?

The class declaration would only contain the member function prototype:
`BankAccount(double);`

- The compiler has to be told that it is part of the class:

```
BankAccount::BankAccount(double startingBal)
{
    balance = startingBal;
}
```

- The main differences to the **inline** definition:
 1. `startingBal` is no longer optional
 2. You need to bring the method into scope with `BankAccount::`
 3. The function must follow the class declaration (usually before the main function).
 4. If you have multiple classes, the class declaration resides in header file `BankAccount.h`, while the member functions reside in a separate file `BankAccount.cpp`.
 5. These files are compiled separately (into `.o` object files) and later linked to the main program's object file.
- What's going on conceptually here?

Separation of interface and implementation.

Using a class in a main function

- We must manage the data flow between the class definition and the main program, where class objects are created and used.
- Example with `BankAccount`:
 1. Create a `BankAccount` named `acct` with 100 starting balance.
 2. Show the current balance.
 3. Deposit 10 and show balance.
 4. Withdraw 50 and show balance.
- Code: Use your previous definition of `BankAccount` and change main.

```
#include <iostream>
using namespace std;

class BankAccount // class declaration
{
private:
    double balance;

public:
    BankAccount(double startingBal)
    { balance = startingBal; }
```

```

void deposit(double amount)
{ balance += amount; }

void withdraw(double amount)
{ balance -= amount; }

double getBalance() const
{ return balance; }

};

int main() // main program - define objects, use class
{
    // create a BankAccount = construct one
    BankAccount acct(100.0);
    // get the current acct balance
    cout << "Current balance: " << acct.getBalance() << endl;
    // deposit some money
    acct.deposit(10.0);
    cout << "Current balance: " << acct.getBalance() << endl;
    // withdraw some money
    acct.withdraw(50.0);
    cout << "Current balance: " << acct.getBalance() << endl;
    return 0;
}

```

```

Current balance: 100
Current balance: 110
Current balance: 60

```

PRACTICE Separate interface and implementation

- Using the BankAccount class provided in the starter code, separate the class interface and the implementation by splitting the code into:
 - class declaration with member data and member function prototypes,
 - definitions of the member function prototypes.
 - When you're done, create a BankAccount named account with a starting balance of 200, deposit 300, withdraw 100, and print the balance each time.
- Sample output:

```

Account starting balance: $200.
Deposited $300. New balance: $500.
Withdrew $100. New balance: $400.

```

- Starter code: tinyurl.com/bank-account-test

```

// The BankAccount class has a balance. It is created with a starting
// balance. You can deposit or withdraw funds. You can get a balance
// printout.
class BankAccount
{
private:
    double balance;

public:
    BankAccount(double startingBal)

```

```

{ balance = startingBal; }

void deposit(double amount)
{ balance += amount; }

void withdraw(double amount)
{ balance -= amount; }

double getBalance() const
{ return balance; }

};

```

- Solution video: https://youtu.be/CyJvZ7nB_IU)
- When you have separated interface and implementation, your files look like this:

```

// BankAccount.h
class BankAccount {
private:
    double balance;
    //...
public:
    double deposit(double);
};

// BankAccount.cpp
double BankAccount::deposit(double amount)
{ balance += amount; }

// Test implementation
int main()
{
    BankAccount b;
    b.deposit(300.);
}

```

Example: A Rectangle class

- Which data members should a Rectangle class have? Which types?

As a geometric object it is fully defined by its width and its length. These should be double.

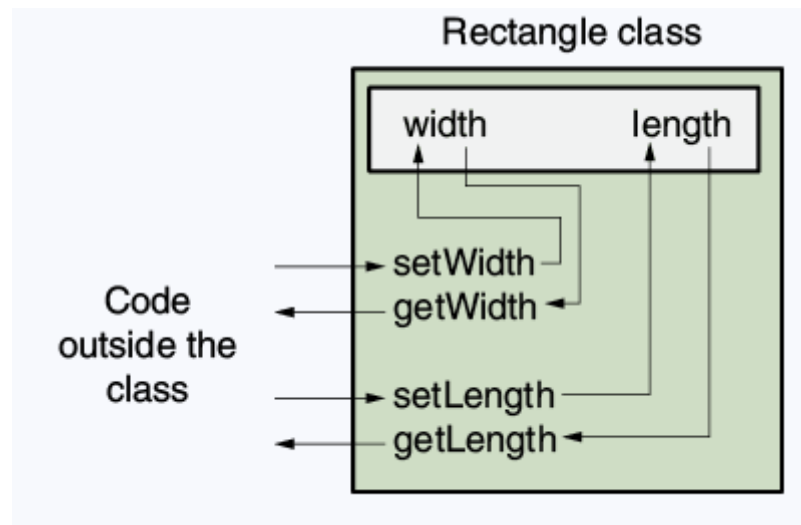
- Which member functions should a Rectangle class have?

- Getters: getWidth, getLength
- Setters: setWidth, setLength

- What return types and arguments should the member functions have?

- Getters: getWidth, getLength <- double (give back member data)
- Setters: setWidth, setLength <- void (change member data)

- In OOP, an object protects its important data by making them private, and provides a public interface to access the data.



PRACTICE Create a Rectangle class

- Using the starter code below, **write a declaration** for the Rectangle class that includes the width and length data members, and the member functions getWidth, getLength, setWidth, and setLength.

Write all member functions as **inline functions** (definitions in the class declaration). Remember to **run your code** as soon as you finished a statement for syntax checks.

Test this class using the main function:

- Create a Rectangle object r with width 4 and length 5.
- Print width and length of the object.

- Sample output:

```
Width = 4
Length = 5
```

- Starter code: tinyurl.com/rectangle-class-starter

```
// Declare and test a Rectangle class
// Header files

// Namespace declaration

// Class declaration

// private member data
// rectangle width
// rectangle length
// public member functions

// set rectangle width
// set rectangle length
// get rectangle width
// get rectangle length
```

```
// Main function

// create a Rectangle object r
// set object width to 4
// set object length to 5
// show object width
// show object length
```

For the solution, see: tinyurl.com/rectangle-class-solution

```
// Declare and test a Rectangle class
// Header files
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle
{
    // private member data
private:
    double width; // rectangle width
    double length; // rectangle length
    // public member functions
public:
    // all member functions are defined inline
    void setWidth (double w) { width = w; } // set rectangle width
    void setLength (double l) { length = l; } // set rectangle length
    double getWidth() const { return width; } // get rectangle width
    double getLength() const { return length; } // get rectangle length
};

// Main function
int main()
{
    Rectangle r; // create a Rectangle object r
    r.setWidth(4.); // set object width to 4
    r.setLength(5.); // set object length to 5
    cout << "Width = " << r.getWidth() << endl; // show object width
    cout << "Length = " << r.getLength() << endl; // show object length
    return 0;
}
```

TODO Home assignment: Complete the Rectangle class

- **Problem:** Use the Rectangle class definition in the starter code. Add the member functions `setWidth`, `setLength`, `getWidth`, and `getLength` as external member functions (outside of the class definition). Add a new member function `getArea` that computes the area of the rectangle. Test the member function with two Rectangle objects called `box1` (12.7 by 14.5), and `box2` (8.5 by 29.3) by comparing their area.
- **Sample output:**

```
Box 1: 12.7 x 14.5
Box 2: 8.5 x 29.3
Box 1 is smaller.
```

- **Starter code:** tinyurl.com/rectangle-assignment - Rectangle class code in one file. Code compiles without output.

```

// Declare and test a Rectangle class
// Header files
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle {
    // private member data
private:
    double width; // rectangle width
    double length; // rectangle length
    // public member functions
public:
    void setWidth (double ); // set rectangle width
    void setLength (double ); // set rectangle length
    double getWidth() const; // get rectangle width
    double getLength() const; // get rectangle length
};

// Member function definitions
//*****
// setWidth assigns its argument to the private member width      *
//*****

//*****
// setLength assigns its argument to the private member length    *
//*****

//*****
// getWidth returns the value in the private member width         *
//*****

//*****
// getLength returns the value in the private member length       *
//*****

```

- **Extension:** We will work in class with separated header and implementation files using Google Cloud shell.

PRACTICE Separate interface and implementation files

- We separated class declaration (interface part) and member function definitions (implementation part) [in an earlier exercise](#).
- Now, we're going to store the declaration in a header file `Rectangle.h`, the definitions in a C++ file `Rectangle.cpp`, and the testing program in a third file, `RectangleTest.cpp`.
- How can we run the test using these three files? Take a guess!
 1. The `Rectangle.h` will be included as a header file in both `Rectangle.cpp` and `RectangleTest.cpp` (which is the main pgm).
 2. `Rectangle.cpp` will be compiled into `Rectangle.o` without linking it (`-c` flag of the `g++` compiler).
 3. `RectangleTest.cpp` will be compiled and linked with `Rectangle.o`.
 4. On Linux there is a utility program called `make` that can execute all of these steps using a config file `Makefile`.
- We're first going to do this manually on Google Cloud Shell, and then with a `Makefile`.

- Actually, you can also do this in OneCompiler.com (add .h and .cpp files with the + button).

PRACTICE Tangle separate files

- Create Rectangle.h

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle {
    // private member data
private:
    double width; // rectangle width
    double length; // rectangle length
    // public member functions
public:
    void setWidth (double ); // set rectangle width
    void setLength (double ); // set rectangle length
    double getWidth() const; // get rectangle width
    double getLength() const; // get rectangle length
    double getArea() const; // compute and get rectangle area
};
#endif
```

- Create Rectangle.cpp

```
#include "Rectangle.h"

// Member function definitions
//*****
// setWidth assigns its argument to the private member width *
//*****
void Rectangle::setWidth (double w)
{
    width = w;
}
//*****
// setLength assigns its argument to the private member length *
//*****
void Rectangle::setLength (double l)
{
    length = l;
}
//*****
// getWidth returns the value in the private member width *
//*****
double Rectangle::getWidth() const
{
    return width;
}
//*****
// getLength returns the value in the private member length *
//*****
double Rectangle::getLength() const
{
    return length;
}
```



```

}
//*****
// getArea computes and returns the area from private data      *
//*****
double Rectangle::getArea() const
{
    return (width * length);
}

```

- Create RectangleTest.cpp

```

#include "Rectangle.h"

// main function
int main()
{
    Rectangle box1;
    box1.setWidth(12.7);
    box1.setLength(14.5);
    cout << "Box 1: "
          << box1.getWidth() << " x " << box1.getLength()
          << endl;
    Rectangle box2;
    box2.setWidth(8.5);
    box2.setLength(29.3);
    cout << "Box 2: "
          << box2.getWidth() << " x " << box2.getLength()
          << endl;
    // Test getArea()
    if (box1.getArea() < box2.getArea())
        cout << "Box 1 is smaller.\n";
    else
        cout << "Box 2 is smaller.\n";
    return 0;
}

```

PRACTICE Compiling and testing a class from multiple files

1. Log into your Lyon Google account and open ide.cloud.google.com
2. Wait until your Linux Virtual Machine has been started (1 min).
3. Ignore the push to use Gemini, the AI assistant, instead just enlarge the terminal window (lower half of the screen).
4. The following commands are useful - enter them after the \$ prompt:

```

$ ll                # alias for `ls -alF` show details of all files
$ cd dir            # change to directory `dir`
$ cd ..             # go one level up
$ nano file         # open `file` in the `nano` editor
$ man cmd           # get documentation on `cmd`
$ g++ file -o out   # compile `file` into object file `out`
$ file file         # give information about `file`
$ clear             # clear the screen
$ cat file          # display the content of `file`
$ rm file           # delete `file`

```

5. Get the three files from GitHub using wget. Enter the following commands at the \$ prompt, then check they're there with ll.

```
wget -O Rectangle.h tinyurl.com/rectangle-h
wget -O Rectangle.cpp tinyurl.com/rectangle-cpp
wget -O RectangleTest.cpp tinyurl.com/rectangletest-cpp
```

6. Execute the following commands to build the executable `rect`, and then run it in the last step:

```
g++ -c Rectangle.cpp
g++ RectangleTest.cpp Rectangle.o -o rect
./rect
```

If any of your `.cpp` or `.h` files cannot be found, add the current directory (`.`) to your `PATH` with: `export PATH="$PATH:."`

7. Use `wget` to get the Makefile from `tinyurl.com/rectangle-makefile` and store it as `Makefile`.

```
wget -O Makefile tinyurl.com/rectangle-makefile
```

8. Let's take a look at it to understand what it does:

```
rectangle: RectangleTest.o Rectangle.o
g++ RectangleTest.o Rectangle.o -o rectangle

Rectangle.o: Rectangle.cpp Rectangle.h
g++ -c Rectangle.cpp

RectangleTest.o: RectangleTest.cpp Rectangle.h
g++ -c RectangleTest.cpp

clean:
rm -f *.o rectangle
```

9. To use the Makefile with `make`, simply run `make` on the Google shell, and then run the executable `./rectangle` to check the outcome. If you run `make` again now, it will not do anything (no file changed):

```
g++ -c RectangleTest.cpp
g++ -c Rectangle.cpp
g++ RectangleTest.o Rectangle.o -o rectangle
```

10. To see `make`'s magic at work, enter the command `touch Rectangle.o`. This will change its timestamp and `make` will get to work. Finally, you can run `make clean` to clean up and remove temporary files.

```
g++ RectangleTest.o Rectangle.o -o rectangle
```

Example: Planning a house (multiple Rectangle objects)

- Constraints: Includes [No description for this link](#) (needs to be tangled and be available as `../src/Rectangle.cpp`, and `Rectangle.h` must also be there). Later, we will see a version with dynamic allocation
- Sample input/output:

```
: What is the kitchen's length? 10
: What is the kitchen's width? 14
: What is the bedroom's length? 15
: What is the bedroom's width? 12
: What is the den's length? 20
: What is the den's width? 30
: The total area of the three rooms is 920
```

- Code:

```
#include <iostream>
#include "Rectangle.cpp"
using namespace std;

//*****
// Function main
//*****
int main()
{
    double number;    // number input
    double totalArea; // total area
    Rectangle kitchen; // kitchen dimensions
    Rectangle bedroom; // bedroom dimensions
    Rectangle den;     // den dimensions

    // Get the kitchen dimensions
    cout << "What is the kitchen's length? ";
    cin >> number; cout << number << endl;    // get the length
    kitchen.setLength(number);                // store in kitchen object
    cout << "What is the kitchen's width? ";
    cin >> number; cout << number << endl;    // get the width
    kitchen.setWidth(number);                 // store in kitchen object

    // Get the bedroom dimensions
    cout << "What is the bedroom's length? ";
    cin >> number; cout << number << endl;    // get the length
    bedroom.setLength(number);                // store in bedroom object
    cout << "What is the bedroom's width? ";
    cin >> number; cout << number << endl;    // get the width
    bedroom.setWidth(number);                 // store in bedroom object

    // Get the den dimensions
    cout << "What is the den's length? ";
    cin >> number; cout << number << endl;    // get the length
    den.setLength(number);                    // store in den object
    cout << "What is the den's width? ";
    cin >> number; cout << number << endl;    // get the width
    den.setWidth(number);                     // store in den object

    // Calculate the total area of the three rooms
    totalArea = kitchen.getArea() +
        bedroom.getArea() +
        den.getArea();

    // Display total area of the three rooms
    cout << "The total area of the three rooms is "
        << totalArea << endl;
    return 0;
}
```

- Testing the tangled program (org-babel-tangle) - cursor right after the command, then C-x C-e with the values

```
cd ../src
g++ rectangle2.cpp -o rectangle2
echo "10 14 15 12 20 30" | ./rectangle2
```

Pointers to class Objects

- A class is a user-defined type. Its objects are regions of memory.
- You can define a pointer that points at a class object. You can then use the -> operator to call member functions (instead of dot .)

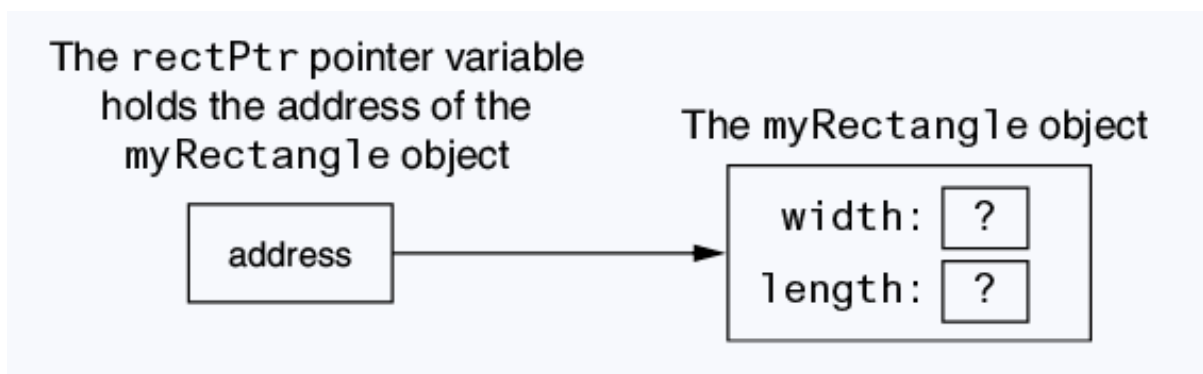


Figure 4: State of the object pointed at before initialization.

- Class object pointers can be used to dynamically allocate objects. Unless you use a **smart pointer**, you have to delete the object when you're done.

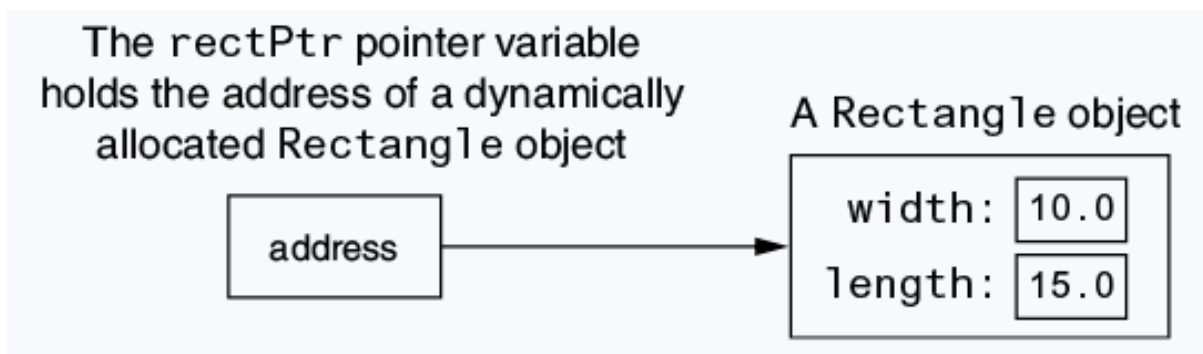


Figure 5: State of the object pointed at after initialization.

- Code example using the Rectangle class:

```
#include <iostream>
#include "Rectangle.cpp"
using namespace std;

int main()
{
    return 0;
}
```

- Same thing with dynamically allocated pointers (new, delete)

```
#include <iostream>
#include "Rectangle.cpp"
using namespace std;

int main()
{
    Rectangle R; // create a rectangle
    Rectangle *rectPtr = nullptr; // create a pointer (initialize it)
    rectPtr = new Rectangle; // dynamically allocate a Rectangle object

    rectPtr->setWidth(11.5); // store width value with the pointer
    rectPtr->setLength(5.8); // store length value

    cout << "Width: " << rectPtr->getWidth() << endl;
    cout << "Length: " << rectPtr->getLength() << endl;

    // must free allocated memory
    delete rectPtr;
    rectPtr = nullptr;

    return 0;
}
```

```
Width: 11.5
Length: 5.8
```

PRACTICE Using pointers to objects [Bonus assignment]

- **Task:** Modify the house building program to dynamically allocate Rectangle objects, kitchen, bedroom, and den.
- **Constraints:** Includes [No description for this link](#) (needs to be tangled and be available as ./src/Rectangle.cpp, and Rectangle.h must also be there).
- **Sample input/output:**

```
: What is the kitchen's length? 10
: What is the kitchen's width? 14
: What is the bedroom's length? 15
: What is the bedroom's width? 12
: What is the den's length? 20
: What is the den's width? 30
: The total area of the three rooms is 920
```

Constructors

- What is a constructor?
- Simple example:

A constructor is a member function that is automatically called when a class object is created. It has the name as the class, and it does not return anything.

- Demo example:

```
#include <iostream>
#include <string>
using namespace std;

// declaration -> Demo.h
class Demo
{
public:
    Demo(); // constructs a Demo object = prototype
    Demo(string);
};

// implementation -> Demo.cpp
Demo::Demo()
{
    cout << "Welcome to the Demo constructor!\n";
}
Demo::Demo(string property)
{
    cout << "I am " << property;
}

// test file -> DemoTest.cpp
int main()
{
    Demo hello;
    Demo goodbye("tired");
    return 0;
}
```

```
Welcome to the Demo constructor!
I am tired
```

PRACTICE Improving Rectangle with a new explicit constructor

- To change the code on Google Cloud Shell, use the nano editor.
- We're going to make two improvements:
 1. Make the code safer against invalid data entry (implementation).
 2. Provide a constructor that initializes a rectangle of zero width and length using an initializer list.
- We'll keep the specification and the implementation apart in a .h and a .cpp file, and we'll test with a Makefile on the shell.
- Rectangle.h (with #include guard) - add new default constructor:

```
// Specification file for the Rectangle class
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle
{
private:
    double width;
    double length;
public:
    Rectangle();           // constructor
    void setWidth(double); // mutator: Width
    void setLength(double); // mutator: Length
    double getWidth() const // accessor: Width
    {return width;}
    double getLength() const // accessor: Length
    {return length;}
    double getArea() const // accessor: Area
    {return width * length;}
};
#endif
```

- Rectangle.cpp (with #include "Rectangle.h") - with data check:

```
#include "Rectangle.h"
#include <iostream>
#include <cstdlib>
using namespace std;

//*****
// The constructor initializes width and length to 0.0
//*****
Rectangle::Rectangle()
//: width(0.0), length(0.0) {} // initialized with a list
{
    width = 0.0;
    length = 0.0;
}
//*****
// setWidth sets the value of the member variable width
//*****
void Rectangle::setWidth(double w)
{
    if (w >= 0)
        width = w;
    else
    {
        cout << "Invalid width.\n";
        exit(EXIT_FAILURE); // or: return 1;
    }
}
//*****
// setLength sets the value of the member variable length
//*****
void Rectangle::setLength(double l)
{
    if (l >= 0)
        length = l;
    else
    {
        cout << "Invalid length.\n";
        exit(EXIT_FAILURE);
    }
}
```

```
}  
}
```

- RectangleTest.cpp (with #include "Rectangle.h") - make a rectangle:

```
#include <iostream>  
#include "Rectangle.h"  
using namespace std;  
  
int main()  
{  
    Rectangle box;  
    // Display rectangle data  
    cout << "Here is the rectangle's data:\n";  
    cout << "Width: " << box.getWidth() << endl  
        << "Length: " << box.getLength() << endl  
        << "Area: " << box.getArea() << endl;  
  
    return 0;  
}
```

- Test the new setup on the command-line:

```
cd ../src  
g++ -c Rectangle.cpp -o Rectangle.o # produce Rectangle.o  
g++ RectangleTest.cpp Rectangle.o -o rect # produce executable  
./rect
```

- From C++11, you can also initialize a member variable in-place, namely when the variables are defined:

```
class Rectangle  
{  
    private:  
        double width = 0.0;  
        double length = 0.0;  
};
```

Passing Arguments to Constructors

- A constructor can have parameters and can accept arguments when an object is created.
- Example: Here is a constructor with parameters for the Rectangle class.

1. Rectangle.h - A Rectangle with undefined width and length

```
Rectangle(double, double);
```

2. Rectangle.cpp - A Rectangle with defined width and length

```
Rectangle::Rectangle(double w, double l)  
{  
    width = w;  
    length = l;  
}
```


Or with an initializer list:

```
Rectangle::Rectangle(double w, double l) :  
    width(w), length(l);
```

3. RectangleTest.cpp: Creating a Rectangle with width and length

```
Rectangle box(10.0, 12.0); // creates 10 x 12 rectangle
```

- You can also pass **default arguments** to the constructor. In the example, a default tax rate of 5% is written into the constructor.

In Sale.h for a Sale class:

```
Sale(double cost, double rate=0.05); // constructor
```

In main:

```
Sale itemSale(cost); // The Sale item has rate = 0.05
```

- What if a constructor has default arguments for all its parameters?

If a constructor has default arguments for all its parameters, it can be called without explicit arguments. It then becomes the default constructor.

In Rectangle.h:

```
Rectangle(double w=0, double l=0); // Rectangle declaration
```

In RectangleTest.cpp:

```
// This works if either there is no default constructor, or  
// your default constructor's parameters are all initialized.  
Rectangle box;
```

- What if all constructors of a class require arguments?

When all constructors of a class require arguments, there is no default constructor. If you don't pass arguments, you get an error.

Author: Marcus Birkenkrahe

Created: 2026-04-11 Sat 07:58